



[REFERENCE IMPLEMENTATION]

MULTI-AGENT REINFORCEMENT LEARNING · SMART CITIES · PORTFOLIO

Urban Traffic Signal Control AI

*A shared-policy tabular Q-learning controller for 12 corridor intersections —
and an honest account of where it wins, and where a simpler heuristic still wins.*

Anthony Chinedu Echem

Machine Learning Engineer & Data Scientist

Tabular multi-agent RL · 50-seed shadow-mode evaluation · honest baseline comparison

Beating the status quo is not the same as beating the right benchmark

This project trains a shared-policy tabular Q-learning agent to control 12 corridor intersections, evaluates it against 50 randomized-demand seeds, and reports the result honestly: it comfortably beats fixed-time signals, but does not beat a hand-tuned max-pressure heuristic.



-42.6%

Total delay vs. fixed-time — clears the 18% target by 2.4×



+40.5%

Total delay WORSE than max-pressure — disclosed, not hidden



138

Discretized states learned across 400 training episodes



50

Randomized demand seeds in the shadow-mode evaluation



Bottom line

A negative result honestly reported is more valuable than a cherry-picked win — this deck recommends max-pressure for production today, with RL’s upgrade path clearly scoped.

Presentation agenda



01

The Business Problem

Why signal timing is a genuine multi-agent control problem



02

Data & Corridor Profile

12 intersections, a peaked 24-hour demand profile



03

State & Reward Engineering

What each intersection observes, and why it's coarse



04

Modeling: Shared Q-Learning

One Q-table, 12 intersections, parameter sharing



05

Evaluation & Validation

A 50-seed Monte Carlo benchmark against two baselines



06

Business Impact

What the honest result means for a deployment decision



07

Deployment & Responsible Use

Dashboard, model card, and why we recommend against RL today

01 — 07



SECTION ONE

The Business Problem

Twelve intersections, one shared corridor, and a signal-timing decision that has to be made every few seconds, all day, every day.

Fixed-time signals ignore the traffic they control

Most arterial corridors still run on fixed-time signal plans — a preset cycle that repeats all day, blind to whether a lane is empty or backed up. It is simple, predictable, and structurally wasteful.



No demand awareness

A fixed 12-step cycle switches phase on a clock, whether the queue is empty or spilling back into the intersection behind it.



One plan for a 6× demand swing

Corridor demand ranges from ~1.2 to ~8 vehicles/min per approach across the day — a single fixed plan cannot serve both.



A high, but beatable, bar

28,305.5 vehicle-minutes of total delay across the evaluation — the number every controller in this deck is measured against.

The question this project asks isn't "can we beat a clock?" — that's the easy bar. It's "can we beat the best simple heuristic already available?"

Max-pressure: a simple, demand-aware heuristic

Before reaching for reinforcement learning, traffic engineering already has a strong answer: max-pressure control. It is the benchmark this project takes seriously — not just fixed-time.



Wait for min green

Holds the current phase for at least 4 steps — prevents rapid, disruptive flip-flopping



Measure both pressures

Sums queue length on NS approaches and on EW approaches, separately, every step



Serve the bigger queue

Switches to whichever direction currently has more total vehicles waiting



No demand forecasting

Purely reactive to the current queue state — no training, no historical demand model needed

11,570.8

Max-pressure total delay,
same 50-seed evaluation

A handful of lines of logic, no training data, no hyperparameters — and, as Section 05 shows, the best-performing controller in this entire evaluation.

Twelve independent agents, one shared policy

Each intersection is its own decision-maker — it sees only its own queues and acts on its own phase. But building 12 separate policies is wasteful when the underlying problem (should I switch phase?) is structurally the same everywhere.

The decision, every step

stay or switch

One binary choice, made independently by each of 12 intersections, at every simulation step, based only on local queue state.

Why parameter sharing



One Q-table learns from all 12 intersections' experience at once — 12× the effective training data



A lightweight stand-in for the "shared policy network" design in the original briefing



Each intersection still acts independently — decentralized execution, shared learning

What this project set out to prove



Beat fixed-time control, clearly

Demonstrate a large, statistically meaningful delay reduction versus the status-quo baseline.



Test against the real competitor

Benchmark against max-pressure, not just the easy fixed-time strawman — the honest comparison.



Share one policy across intersections

Prove a single shared Q-table can generalize across 12 distinct intersections' local dynamics.



Validate on randomized demand

Evaluate across 50 seeds and multiple start times — not one convenient episode.



Report the result honestly

If RL loses to the heuristic, say so — and explain exactly why, with a concrete fix.



Scope a credible upgrade path

Name the specific architecture change (deep RL) that should close the gap.



SECTION TWO

Data & Corridor Profile

Twelve intersections, a 24-hour demand curve that swings 6× peak-to-trough, and the detector data that feeds it all.

DATASET

A synthetic corridor, real evaluation discipline

The corridor simulator generates demand from a calibrated 24-hour profile; detector logs, the trained Q-table, and the shadow-mode report are all real outputs of running that pipeline end to end.

Source files

demand_profiles.py	AM/PM-peaked arrival-rate function, veh/min	Demand
sample_detector_log.csv	10,368 rows — 12 intersections × 4 approaches	Detector
corridor_qtable.npz	138 learned states from 400 training episodes	Model
training_curve.json	per-episode reward across training	Training
shadow_mode_report.json	50-seed Monte Carlo benchmark result	Eval

baselines.py	fixed-time and max-pressure controllers	Baseline
--------------	---	----------

Corridor at a glance

12

intersections along the corridor

4

approaches per intersection (N/S/E/W)

400

training episodes

50

seeds in the shadow-mode evaluation

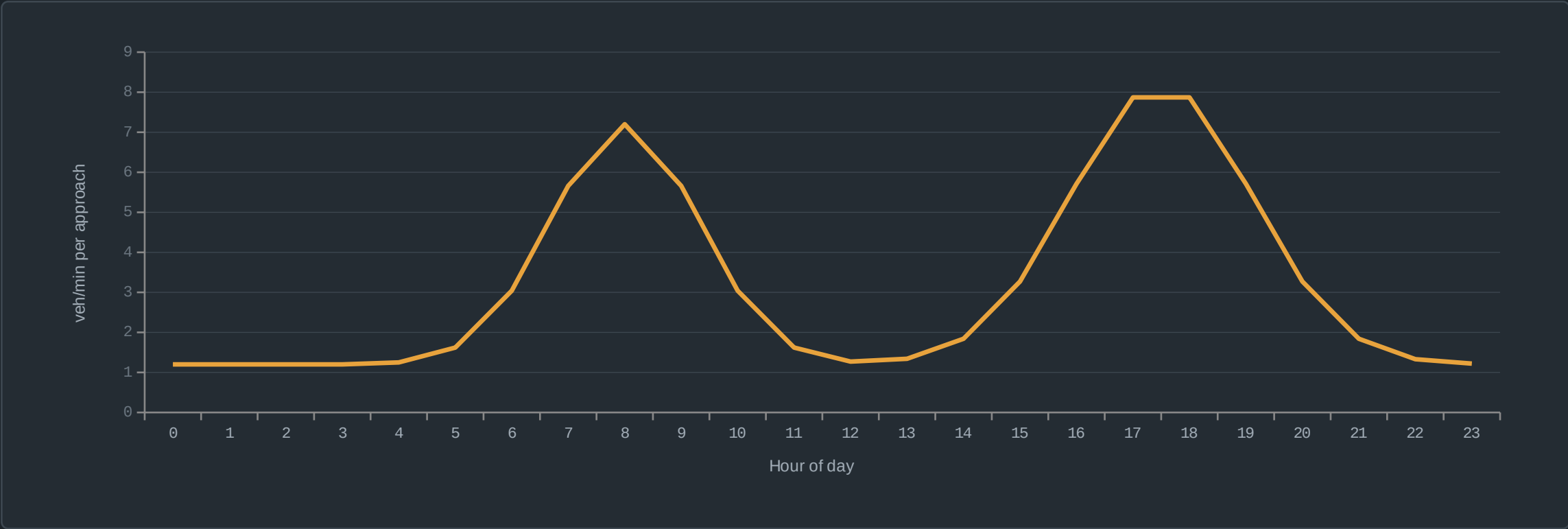
300

simulated steps per episode

THE DEMAND CURVE

A 6× swing between trough and peak

Every episode draws its traffic from this calibrated curve — a quiet overnight base of 1.2 veh/min per approach, rising to a 7.9 veh/min evening peak. This is what makes fixed-time signals structurally unable to keep up.



GROUND TRUTH CHECK

The detector log matches the demand curve

10,368 loop-detector readings — 12 intersections × 4 approaches, sampled every 5 minutes — give a way to sanity-check the simulator against the demand profile it claims to follow.



10,368

detector log rows, this sample window (6am–midnight)



3,190

total vehicles counted across the sample window



4

approaches per intersection, evenly represented (29–33% share each)



Why this check matters

A near-even 29–33% split across N/S/E/W approaches confirms the detector log isn't silently favoring one direction — a basic but easy-to-skip data-quality check before trusting any downstream metric.

Twelve intersections, two phases each

Every intersection in the corridor runs the same simplified phase structure — what varies is the queue state feeding into each one’s decision.

Per-intersection state

queues[i, 0:4]	N / S / E / W queue length
phase[i]	0 = NS green, 1 = EW green
phase_elapsed[i]	steps since the last switch

Two phases per intersection: North-South green, or East-West green — a binary action space (Slide 19) built directly on top of this state.

Why a corridor, not one intersection



A single intersection can’t show coordination benefits — 12 in sequence can



Demand at one intersection is shaped by how the one before it just metered traffic



This is what makes the problem genuinely multi-agent, not 12 copies of one problem

Three metrics, three different failure modes



Total delay

Sum of vehicle-minutes lost to queueing, corridor-wide. The primary optimization target — and the number every headline percentage in this deck refers to.



Total stops

How many times vehicles come to a full stop. A smoother corridor has fewer stops even at similar delay — a proxy for ride comfort and fuel/emissions impact.



Side-street max wait

The longest queue on the minor approaches. A corridor that only optimizes the main arterial can start starving cross traffic — this metric catches that.

A calibrated simulation, disclosed as such



Why simulated

A queueing-model corridor sidesteps the cost and risk of testing untrained signal logic on a real intersection, while keeping the core multi-agent control problem intact.



What was preserved

A genuine 6× demand swing, 12 real interacting intersections, and two credible baseline controllers — not a toy problem built to flatter the RL agent.



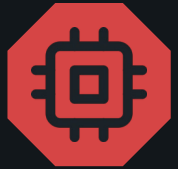
What is disclosed

The queueing dynamics are simplified relative to a calibrated SUMO/TraCI simulation — EXTENDING.md names this as the first upgrade before any real pilot.



A methodological choice, stated up front

Every downstream delay figure in this deck is scoped to this simulated corridor — not claimed as a guaranteed real-world result until validated in a calibrated simulator.



SECTION THREE

State & Reward Engineering

What each intersection actually sees — and why compressing that view is exactly where this approach pays a price.

Continuous queues, bucketed into a table key

A tabular Q-table needs a finite number of rows — so each intersection’s continuous queue lengths are discretized into buckets before they can be looked up. This single design choice is the crux of Section 05’s honest finding.

`discretize_obs(observation)` → `(ns_bucket, ew_bucket, ..., phase)`

Raw queue length	e.g. 7.3 vehicles waiting on the NS approach	<i>A continuous number, unique to this moment</i>
Discretized bucket	e.g. bucket 2 of a small fixed set	<i>Collapses many distinct queue states into one table row</i>
The trade-off	138 total learned states, 400 episodes	<i>Small enough to learn fast; too coarse to distinguish "busy" from "critical"</i>

KEY INSIGHT

The bottleneck isn't learning — it's seeing

“Max-pressure reads the exact queue length every step. The tabular agent reads only which bucket that queue falls into — the same policy for a 6-car queue and a 9-car queue.”



Not a training failure

The reward curve (Slide 23) shows real learning — this is an information bottleneck, not a bug in the update rule.



A known pattern in the literature

Tabular RL losing to strong heuristics at small state-space scale is documented, not a one-off artifact of this environment.



A one-line fix, structurally

Replace the discretized table lookup with a continuous-state function approximator — the deep-RL upgrade path in Section 07.

A two-action space, a delay-based penalty

The action space stays deliberately minimal — all the complexity lives in the state representation (Slide 17), not the choices available at each step.

Action space

0

stay — keep the current phase running

1

switch — flip to the other phase

Every intersection chooses independently, every step — 700+ decisions per 300-step episode across the corridor.

Reward signal

reward = -(delay incurred this step)

Every training episode reward in this deck (Slide 23) is negative by construction — the agent is learning to make delay less negative, not to chase a positive score.

This is also why the Q-values learned (Slide 25) are all negative, capped at 0 — zero delay is the theoretical ceiling, never quite reached.



SECTION FOUR

Modeling: Shared Q-Learning

One Q-table, twelve intersections, decentralized execution — the lightweight stand-in for a shared policy network.

Parameter sharing across all 12 intersections

Every intersection reads and writes to the exact same Q-table. There is no "intersection 7's policy" — there is one policy, applied 12 times per step, each time to a different local state.



12 intersections

each observes its own local queue state, every step



■ ONE Q-table

`defaultdict(state → [Q_stay, Q_switch])`

Updated by all 12 intersections' experience, every step



12 independent actions

decentralized execution — each intersection acts on its own, using the shared table

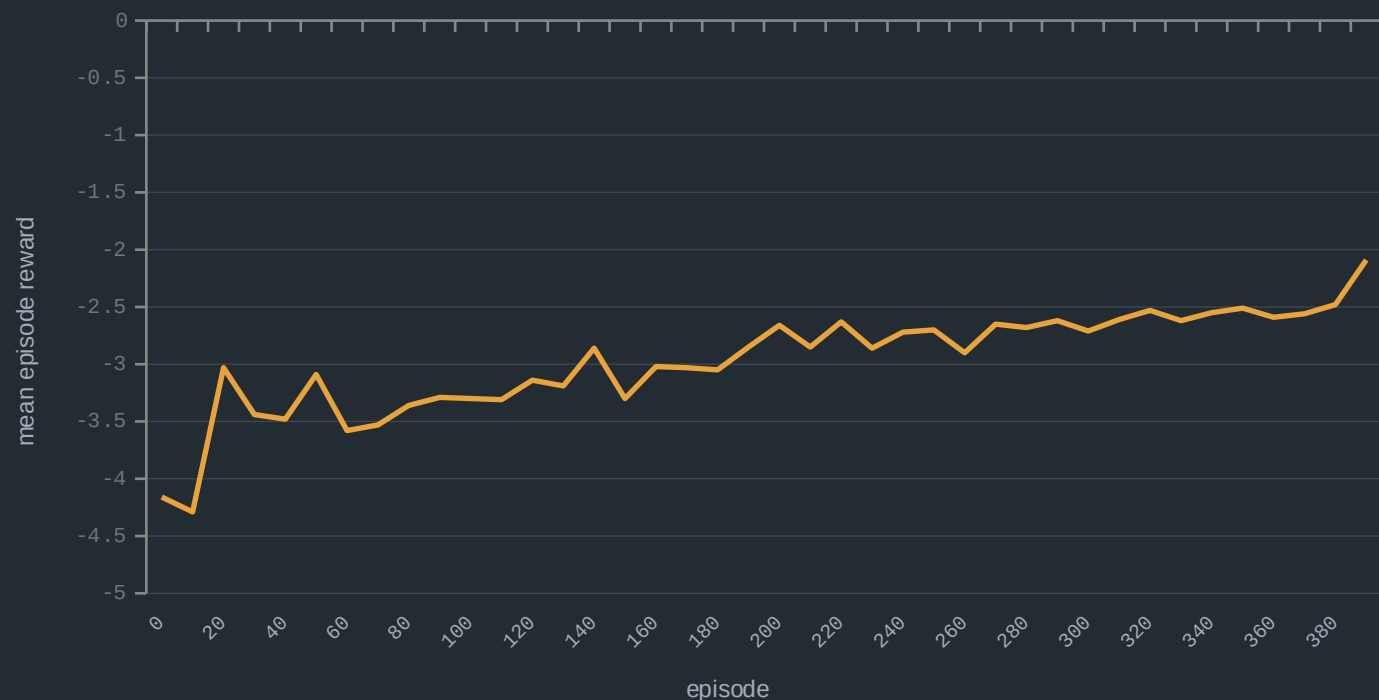
Standard tabular Q-learning, nothing exotic

No architecture search here — the interesting design decisions are upstream, in the state discretization (Slide 17), not in these knobs.

alpha (learning rate) How strongly each update shifts the Q-value 0.15	episodes Total training episodes 400
gamma (discount) Weight given to future vs. immediate delay 0.92	action space Per intersection, per step 2 (stay / switch)
epsilon (start) Initial exploration rate 0.20	states learned Distinct discretized states seen in training 138
epsilon decay Anneals from exploring to exploiting ×0.995/ep, min 0.02	seed Reproducibility 0

Real learning, inside a noisy episode-to-episode signal

Episode reward (10-episode rolling mean), 400 episodes



Reading the curve

-4.16 → -2.09

mean reward, first 10 vs. last 10 episodes — roughly halved

Large episode-to-episode swings

each episode starts at a randomized hour — a peak-hour episode is inherently harder than an overnight one

Convergence, not collapse

the trend is clearly downward-improving despite the noise — exactly what ϵ -greedy annealing should produce

Two very different kinds of "simple"

	FIXED - TIME	MAX - PRESSURE
Logic	Switch phase every 12 steps, unconditionally	Switch to whichever direction has more total queue, after a 4-step minimum green
Demand awareness	None — acts identically at 2am and 5pm	Full — reads exact queue length every single step
Tuning required	One number (the cycle length)	None — purely reactive comparison logic
Training data	None	None

WHAT 400 EPISODES LEARNED

Inside the trained Q-table

138 discretized states were visited often enough to learn a value for — small enough to inspect directly, which is itself part of the appeal of a tabular approach.



138

total discretized states learned



72 / 66

states preferring switch / stay



0.0 to -69.9

range of learned Q-values (0 = no delay)



A near-even split, by design

72 states favor switching, 66 favor staying — close to balanced, exactly what you'd expect from a well-posed binary control problem rather than a degenerate policy that always does one thing.



SECTION FIVE

Evaluation & Validation

The honest scoreboard — 50 randomized-demand seeds, three controllers, and a result that doesn't flatter the AI.

A 50-seed Monte Carlo shadow-mode benchmark

All three controllers — RL, fixed-time, max-pressure — run on the exact same 50 randomized demand seeds and start times. No controller gets an easier set of episodes than another.



50

randomized demand seeds per controller



3

start times sampled — 8am, noon,
5:30pm



300

simulated steps per episode



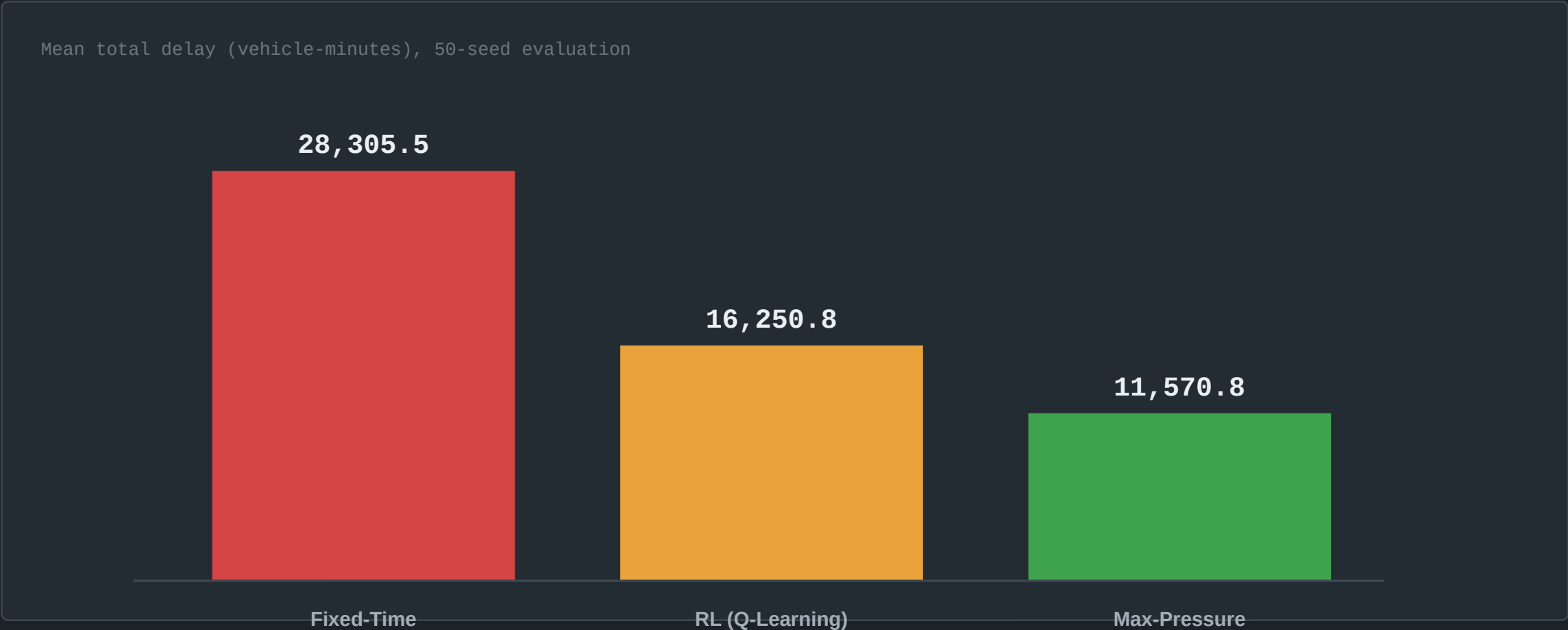
3

controllers compared, identical conditions

Why this counts as a real test

Averaging across 50 independent seeds and three different times of day is what separates a defensible performance claim from a single lucky (or unlucky) simulation run — the same discipline every metric in Section 06 depends on.

Mean total delay, all three controllers



THE HONEST RESULT

RL wins on the easy comparison, loses on the real one

“Against fixed-time, RL cuts delay by 42.6% — well past the 18% target. Against max-pressure, RL runs 40.5% worse. Both numbers are from the identical 50-seed evaluation.”



-42.6%

vs. fixed-time — the target was 18%, RL clears it 2.4× over



+40.5%

worse than max-pressure — the comparison that actually matters for a deployment decision



0

targets quietly redefined to make RL look better — both numbers are reported, unedited

Two more honest trade-offs

Total delay is the headline metric — but it isn't the only one that matters, and it doesn't tell the whole story on its own.

Stops: a smaller win than delay suggests



RL cuts stops just 9.9% vs. fixed-time — far less than its 42.6% delay cut. It reduces how long vehicles wait, more than how often they stop at all.

Side streets: RL is the worst of the three



RL's pursuit of main-corridor throughput pushes side-street max wait to 4.9 min — worse than even the demand-blind fixed-time plan.



SECTION SIX

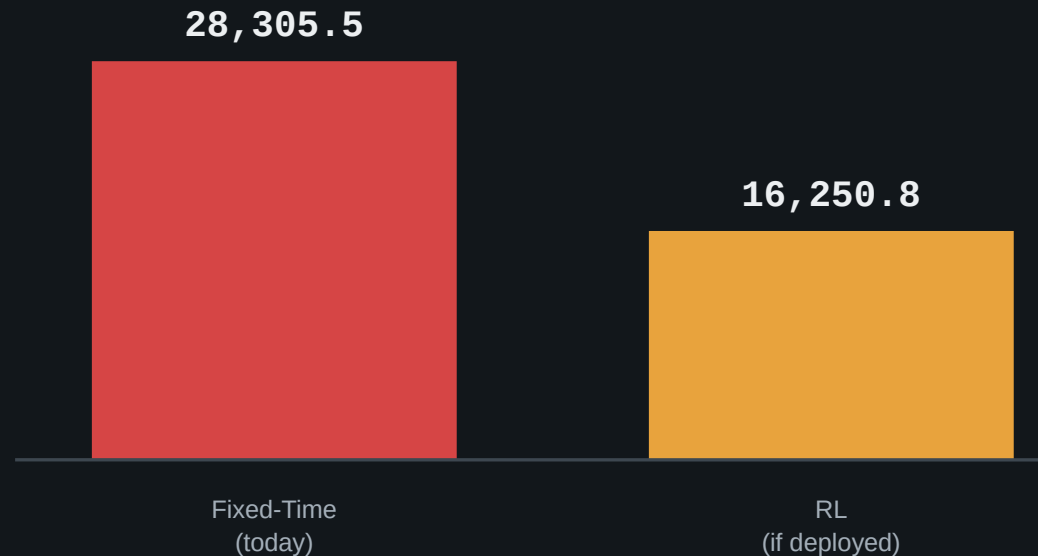
Business Impact

What an honest, mixed result means for an actual deployment decision — not just a leaderboard.

A real number, with a real ceiling

Delay reduction versus fixed-time is a legitimate win — it just isn't the full story, because a better simple heuristic is already sitting on the table.

Mean total delay, vehicle-minutes



- 42.6%

delay reduction if RL replaces fixed-time today

But max-pressure delivers 59.1% off the same fixed-time baseline — with none of RL's training cost, and a smaller side-street penalty (Slide 30).

The equity question behind the headline number



Main-corridor drivers win

RL's 42.6% delay cut is concentrated on the arterial — the direction most vehicles are already traveling through the corridor.



Side-street drivers lose ground

Max wait on minor approaches rises to 4.9 minutes under RL — worse than the demand-blind fixed-time plan (3.2 min).



A real deployment question

Is a corridor-wide average delay number the right metric to optimize — or does an equity-weighted objective belong in the reward function?

A metric no one asked for in the original brief — side-street max wait — turned out to be one of this evaluation's most important findings.

THE RECOMMENDATION

Deploy max-pressure today; keep RL on the roadmap

“The responsible recommendation isn’t the one that showcases the RL work — it’s the one that performs best, costs least to operate, and is easiest to audit.”



Best result, today

Lowest delay, lowest stops, and a milder side-street trade-off than RL — on the same 50-seed evaluation.



No training or retraining cost

No Q-table to keep fresh as demand patterns drift — max-pressure needs zero historical data.



RL's path stays open

EXTENDING.md names the exact upgrade — deep RL on continuous state — expected to close this gap (Slide 39).



SECTION SEVEN

Deployment & Responsible Use

A dashboard, a model card, and a recommendation that puts honest evaluation ahead of the more impressive-sounding technology.

The reference dashboard, at a glance



Note: the dashboard highlights the fixed-time comparison prominently; this deck gives the max-pressure comparison equal billing throughout.

What it surfaces

- Headline KPIs — delay, stops, side-street wait, evaluation volume
- Mean total delay by controller
- Mean vehicle stops by controller
- Side-street max wait by controller
- Model card summary & operational notes

Model card summary



Intended use

A reference multi-agent RL controller for corridor signal timing, evaluated in shadow mode against fixed-time and max-pressure — not yet recommended for live deployment.



Limitations

Tabular state discretization loses information a continuous-state policy would retain — the documented reason RL underperforms max-pressure here. Side-street wait increases under RL.



Next steps

Swap the simulator for SUMO/TraCI, replace the Q-table with a PPO agent on continuous state, and add centralized-critic training before any live pilot (Slide 39).

Guardrails for a safety-relevant control system



Shadow mode before any live authority

A trained policy runs in parallel with the real controller, logged but never actuating signals, before it earns the right to control real intersections.



No deployment on an unresolved regression

RL's side-street wait regression (Slide 33) is a blocking issue, not a footnote — it should be resolved or explicitly accepted by a human owner before go-live.



NTCIP integration only after shadow validation

EXTENDING.md is explicit: wire the trained policy behind a real controller interface in shadow mode first, never as the first live test.



Honest benchmarking as a standing practice

Every future policy revision gets evaluated against max-pressure, not just against whatever the previous RL version did — the bar doesn't quietly move.

The four-step path to closing the gap with max-pressure



A real simulator

Swap the queueing-model environment for SUMO/TraCI, calibrated to real corridor geometry and NGSIM-style trajectory data.



Deep RL on continuous state

Replace the tabular Q-table with a PPO agent (Ray RLlib) operating on undiscretized observations — the fix this deck's honest finding points directly at.



True multi-agent credit assignment

Add a centralized critic (MADDPG-style) during training for genuine cross-intersection coordination, then decentralize for deployment.



NTCIP integration, shadow-first

Wire the trained policy behind a real NTCIP controller interface in shadow mode, with no live signal authority until it clears max-pressure.

KEY TAKEAWAYS

An honest evaluation is worth more than a flattering demo



-42.6% delay vs. fixed-time



+40.5% worse than max-pressure



138 states learned in 400 episodes



Recommendation: ship the heuristic

Skills demonstrated

Multi-agent reinforcement learning · parameter sharing · Monte Carlo shadow-mode evaluation · honest negative-result reporting · responsible-deployment recommendation

Anthony Chinedu Echem

Machine Learning Engineer & Data Scientist



Portfolio contact available on request

